MDL-SPAM: A Multi Model Deep Learning Framework for Email Spam Detection Using VGG-16 and LSTM

Lohith J J[1], Arnav Purushotam[1], Anirudh Mullangi[1] and Arnav Sharma[1]

[1*]Dept. of CSE(AI&ML), Nagarjuna College of Engineering & Technology .

[2]Dept. of Computer Science and Engineering, B.M.S.College of Engineering.

[3]Dept. of Computer Science and Engineerin, B.M.S.College of Engineering.

Contributing authors: lohithjj@gmail.com ; arnavp.cs20@bmsce.ac.in; anirudh.cs20@bmsce.ac.in; arnav.cs20@bmsce.ac.in;

Abstract

With the exponential increase in internet users, the issue of email spam has also grown substantially. A significant portion of emails being circulated contain unwanted and irrelevant content that doesn't pertain to the recipient's interests. In recent times, spammers have devised a novel tactic, which involves embedding spam content within images, making traditional methods that analyze the email's textual content ineffective. This paper presents an innovative approach to combat email spam by utilizing an ensemble of machine learning models capable of detecting spam in both the text and image components of an email. To identify spam within the images, a transfer learning model based on VGG16 (Visual Geometry Group) was used. The dataset used for training this model is less biased than the Dredze image spam dataset, as it includes images containing a mixture of graphics and text, the Dredze dataset primarily consists of text-based images. For the text portion of the email, an LSTM (Long Short-term

Memory) model was trained to identify spam. Finally, a fusion model was developed to combine the results obtained from the two aforementioned models. This model calculates the probability of spam presence in the email. Remarkably, the combined approach achieved an impressive accuracy of 95.43% and precision of 94.12% when compared to the ground truth, effectively countering email spam.

Keywords: Ensemble model; LSTM; VGG16; Spam detection, machine learning.

1 Introduction

Email is a popular method of social interaction and communication. Billions of individuals utilise email communication on a global scale for social, professional, and personal reasons. Unfortunately, email's widespread use has made it a prime target for scammers who want to use it as a manipulative means of spreading potentially harmful information. Spam is the most prevalent and harmful type of attack that can be made on emails[1]. Bulk unsolicited message distribution is referred to as spam or junk email. Often, malicious phishing URLs can be found in spam emails. As a result, it becomes necessary to suggest a careful approach to detect or identify such spam emails. This work proposes a machine learning-based spam detection system that uses a series of machine learning algorithms to assess if a given email is spam or not.

'Ham' is the term for emails that are not spam. A typical email subscriber receives between 40 and 50 emails daily. Spammers make money on a personal and institutional level each year, earning just about 3.5 million USD from spam[2]. Because of this businesses offer a variety of tools and techniques to assist network users in identifying spam emails. filtering techniques have been set up by organisations to discover spam emails by putting up rules and modifying firewall settings. According to MessageLabs figures, network e-mail traffic had just 8% of spam in 2001, but by 2005 it had increased dramatically to 70% [3], demonstrating the high level of risk of internet technology. According to Kaspersky Lab's spam analysis report, a credible security company, the average global spam proportion in all emails in 2017 was as high as 56.63% or more[4]. This incident demonstrates how spam is causing issues for online users and filling up the entire network. Spam email is an issue that affects about 77% of email traffic globally and is becoming worse every year[5].

Most spam emails, including phishing and marketing spam, historically come from the United States[6]. As the pattern spreads to other regions of the world, Russia accounted for more than 20% of all spam transmitted globally in the first half of 2020[7]. Germany and the United States share a tie for second with 9.41% and 9.64% of the global spam, respectively[8].

Email spam may contain viruses, rats, and Trojans. They are used by attackers mostly to persuade users to use online services[9]. They might send spam emails with several file attachments and URLs to malicious and spammy websites, which could lead to data breaches, financial fraud, and identity theft[10]. These factors make email management and the classification of spam emails essential for organisations to boost productivity and decrease financial losses. Learning-based classifiers are widely employed to identify spam today. The learning-based category identification method makes the assumption that spam emails exhibit a particular set of characteristics that distinguish them from legitimate emails[11]. Machine learning makes it simpler as it automatically learns to distinguish between spam emails and actual emails (ham), then it applies the instructions learned to the incoming emails[12].

2 Related Works

The problem of spam detection in digital communication platforms has been a subject of significant research and development. Researchers have explored various methods and techniques to combat the growing threat of spam messages and accounts. Machine learning methods have been extensively employed for spam identification ever since they were first developed. This research topic has grown even more due to image spam. Strong picture spam detection algorithms have been produced because of combining image processing and machine learning approaches.

In [13] the focus was on reviewing some of the key approaches and studies that are relevant to the proposed ”SMSAD” framework for spam message and spam account detection. While existing research offers valuable insights and a range of methods for spam detection, the proposed ”SMSAD” framework aimed to build upon these foundations and address specific limitations. By integrating machine learning, deep learning, and graph-based analysis, the framework aimed to achieve enhanced accuracy and effectiveness in detecting spam messages and accounts across diverse digital platforms.

In the paper [14], the authors focus on Natural Language Processing (NLP) and its growing importance in representing and analysing human language computationally. They highlight NLP's wide range of applications in fields such as machine translation, email spam detection, information extraction, summarization, medical applications, and question answering.

A feature set was created by Annadatha et al. [15] utilising 21 image attributes. Based on how much each feature aids in (Support Vector Machine) SVM classification, a weight is assigned to it. They ran several tests involving feature selection and feature deletion based on these weights. The accuracy on the Dredze dataset increased by 9% because of the feature set's construction using much more features as compared to [16]. A fresh internal dataset was also created to test their SVM classifier.

The authors of the study [17] have highlighted certain email header components that can be used to recognise and classify spam communications with accuracy. Based on how efficiently they identified spam communications, these traits were selected. As a

result of this study, a universal spam message detection system may be recommended for all the major email providers. It also unifies all the capabilities found in Hotmail, Gmail, and Yahoo mail.

In a different study, [18], authors proposed a novel deep learning (DL)-based method for identifying Twitter spam. Author employs both the text of tweets and user meta-data (such as account age, follower count, and other information) to identify spammers. The authors of [19] compare standard machine learning methods for review classification. To retrieve semantic information, the authors of this study proposed an attention- and bidirectional Long Short Term Memory (LSTM)-based method. In [20], The authors offered a cutting-edge technique for distinguishing between authentic and fake content. The technique used is called a horse herd meta Heuristic Optimisation Algorithm (HOA). The discrete algorithm is constructed using the continuous HOA. After that, the algorithm's inputs were changed to be opposition-based and multi-objective. The evaluation's findings show that the suggested method outperforms a variety of machine learning techniques, including K Nearest Neighbour Algorithm (KNN) [21, 22], Naive Bayes, SVM multilayer perceptrons, KNN, and KNN-grey wolf optimisation. Researchers in [23] used LSTM to focus on the tones of news reports. Nevertheless, due to financial limitations, they were unable to constantly monitor the credibility of the news sources they used.

2.1 Our Contribution

Our work proposes a method to detect spam in both the text and image components of an email, thereby preventing spammers who bypass text spam detection systems by embedding text within images. We also propose a new image spam detection dataset, this is an improvement over the Mark-Dredze dataset which mostly includes images with spam text embeddings. Our dataset comprises of a wide range image spam such as advertisement posters, images with text spam embeddings and other types of advertisement images. This reduces the bias within the dataset, thereby making the model more robust.

2.2 Organisation of the paper

The rest of the paper is organized as follows. Section 3 briefs the architecture and framework of the image spam detection model and the text spam detection model. Section 4 explains the details of the Fusion model which helps in the combining of results from the image spam detection model and the text spam detection model. Section 5 is dedicated to showcasing the results obtained from each of the above systems, and finally section 6 provides the conclusion.

3 Methodology

Our model classifies data as either spam or ham (not spam). The spam filtering system can be used to filter spam which consists of only text or only images or a hybrid of both. We propose a framework to handle all these scenarios. Fig. 1 is graphical

depiction of the framework. The fusion model architecture comprises of 4 components.
Data pre-processing, Text-classification model, Image classification model and the Fusion model:
1. Image and text are separately pre-processed from the input and converted into image and text datasets respectively while adhering to the format in order to be fed into their deep-learning models.
2. An LSTM model is used to process the text obtained from the input and return a probability value $P_{Text,}$ which states the probability of spam in the input text data.
3. A Convolutional Neural Network (CNN) model is used to process the images obtained from the input and return a probability value P_{Image} which states the probability of spam in the input image data.
4. A Fusion model consisting of deep layers and logistic regression is used to finally classify the presence of spam or junk in the input email data. We use the dropout principle to set the matching model output probability value p to 0.5 for inputs that comprise text or picture data. This model takes a vector (P_{Image}, P_{Text}) as input and returns an output P_{Total} which is the total probability of the presence of spam in the input. The following sections will explain in detail about the core principles, mathematics and architecture of each of the models used. The pre-processing technique applied and the accuracy obtained using the model. Graphs were plotted for each of these models for visually representing the results obtained.

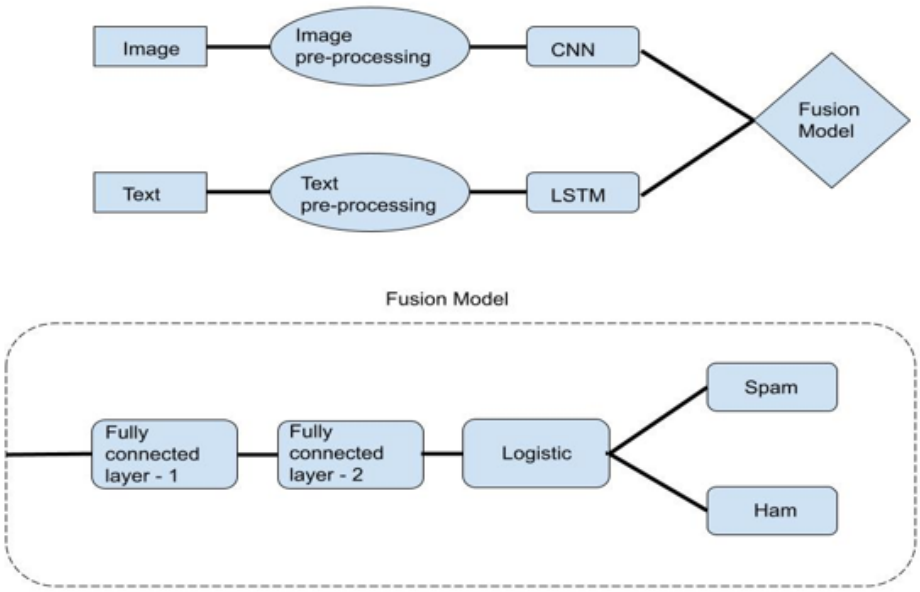


Fig. 1: MDL-SPAM Framework

3.1 Transfer Learning

Transfer learning was used to train our model due to basic computing resources and a relatively small dataset [24]. Using this strategy, a model that has been trained for one task is put to use on another related task. In transfer learning, the model weights that were learned during training on the first task are used as a starting point, and the model is then fine-tuned on the second task using additional labelled data. It adjusts the model weights to better fit the characteristics of the new task, while still leveraging the knowledge learned from the first task [25].

3.2 Text-classification model: LSTM Model

To detect spam consisting of text we have selected the LSTM Model. Recurrent neural networks (RNNs) with Long Short-Term Memory (LSTM) may recognise long-term dependencies in sequential data [26]. They are helpful for activities that require processing data sequences, like time series forecasting, speech recognition, and language translation. LSTMs are composed of cells, which contain gates that control the flow of information [27]. The gates enable the selective recall or deletion of prior knowledge, which helps the network to better handle long-term dependencies. Internal states also known as cell states enable us to store information across various steps.

3.2.1 Mathematics of the LSTM Model

The core mathematics behind LSTMs involves the use of matrix operations to compute the hidden state of the LSTM at each time step [28]. The hidden state is an overview of the previous input and hidden state, and it is used to make a prediction or decision at each time step. The matrix operations used in LSTMs involve the use of linear algebra, including matrix multiplication and element-wise operations

An LSTM can be represented by the following set of equations:

Input gate: $i(t) = \sigma(W_i * x(t) + U_i * h(t-1) + b_i)$

Forget gate: $f(t) = \sigma(W_f * x(t) + U_f * h(t-1) + b_f)$

Output gate: $o(t) = \sigma(W_o * x(t) + U_o * h(t-1) + b_o)$

Cell state: $c(t) = f(t) * c(t-1) + I(t) * \tanh(W_c * x(t) + U_c * h(t-1) + b_c)$

Hidden state: $h(t) = o(t) * \tanh(c(t))$

Output: $y(t) = W_y * h(t) + b_y$

In these equations, x (t) represents the input data at time step t, h (t) represents the hidden state of the LSTM at time step t, and y (t) represents the output of the LSTM at time step t. The parameters W, U, and b are the weights and biases of the LSTM, and the sigmoid and Relu functions are activation functions [29]. The LSTM's memory cell is controlled by the input gate, forget gate, and output gate, and information is

stored and propagated through the LSTM using the cell state and hidden state. The output of the LSTM is computed based on the hidden state at each time step. In our framework, the input is an email with text data T. To process this input, it needs to be transformed into a word vector x. The word vector x is a sequence of word embeddings, represented as (x1, x2, ..., xl), where each xi is an n-dimensional vector that represents the i-th word in the document T. To create the word vector x, the words in the document T are converted into their corresponding word embeddings. These word embeddings are pre-trained models that map words into a continuous vector space. The resulting matrix X represents the document T, with each row representing a word vector xi. The matrix X has dimensions I x n, where I is the maximum length of the document and n is the dimensionality of the word embeddings. In this case, the maximum length I is set to be less than or equal to 500. Once the document T has been embedded into the word vector x, a logistic sigmoid function is applied to each element of the vector x. The logistic sigmoid function maps any real value to a value between 0 and 1. This step is used to determine the categorization probability value for the text segment, specifically whether it is classified as spam or not.

3.2.2 Text dataset

A 5-fold cross-validation technique was utilized to obtain the results of the LSTM model [30]. A total of 5,172 entries consisting of either ham or spam were sourced from the Email spam collection dataset of Kaggle.com (figure 2 is an example of the data entries which were taken as input for the model). The dataset consisted of 87% of ham and 13% of spam. The entire dataset was divided into 5 folds and train-test split of 0.2 was maintained each time. When fold 1 was used a test dataset it consisted of 1,034 entries and the remaining 4,457 entries were grouped into the remaining folds and were used as training datasets. This process was continued when folds 2, 3 and 4 were used as test datasets respectively. When fold 5 was used a test dataset it consisted of 1,112 entries and the remaining 4,060 entries were grouped into the remaining folds and were used as training datasets.

3.2.3 Text Pre-processing

After collecting data it has to be pre-processed. The goal is to clean and prepare the text data in a format that can be easily and effectively consumed by the LSTM model [31].

The following steps were used for pre-processing the text data. 1. Tokenization to break the text into individual words or sub words, which was done using a tokenizer. 2. Removing of stop words, these are common words in a language that do not provide much meaning, such as ”the,” ”a,” and ”an.” These words can be removed to reduce the size of the vocabulary and focus on the more important words in the text [32].
3. Lowercasing to convert all the text to lowercase which helped to reduce the size of the vocabulary and avoid treating words with different capitalization as separate entities [33].
4. Stemming was used to reduce words to their base form, as it helps to reduce the

Table 1: LSTM Model architecture

Layer (type)	Output Shape	Number of Parameters
Embedding_1 (Embedding)	(None, 79, 16)	125392
Bidirectional_1 (bidirectional)	(None, 800)	1334400
Dropout_12 (Dropout)	(None, 800)	0
Dense_21 (Dense)	(None, 20)	16020
Dropout_13 (Dropout)	(None, 20)	0
Dense_22 (Dense)	(None, 1)	21
Total params: 1.475.833		
Trainable params: 1,475,833		
Non-trainable params: 0		

size of the vocabulary and improve the generalization of the model.
5. A word dictionary was created to maintain a record of all the words in the input and their frequencies.
After finishing these steps feature extraction process was started and the classifiers were trained.

3.2.4 Architecture of LSTM Model

The Text classification model consists of a beginning embedding layer that learns an embedding for each word in the training dataset and is initialised with random weights. Afterwards, a dropout layer helps reduce overfitting by changing input units to 0 at random with a frequency of rate at each step during the training period [34]. As a result, the trained model works as an ensemble model consisting of multiple neural networks. The model is now coupled with an LSTM layer having number of units equal to the embedding layer dimensions, return sequences is set to True so that the following LSTM layer has a three-dimensional sequence input, dropout is set to a value of 0.2 to probabilistically eliminate recurrent connections to LSTM units during network training, which enhances model performance. Table 1 depicts the architecture of the LSTM Model used.

Recurrent dropout is set to 0.2. The bias vector and kernel weights matrices both undergo regularisation using the L2 regularizer, which has a regularisation value of 0.005. The input's spatial dimensions are now collapsed into the channel dimension by the addition of a flatten layer. This is followed by the addition of a dense layer with 512 nodes and L2 regularizer with a regularization value of 0.001 is used as both the kernel and bias regularizer functions

The 'ReLU' activation is used as the activation function for this layer On a positive input, the rectified linear activation function, or ReLU for short, will output the input directly; on a negative input, it will output zero. Models can learn more quickly and perform better because to the rectified linear activation function [35].

A dropout layer is now added to prevent overfitting followed by another dense layer with 8 nodes and same regularizer and activation functions. This is followed by one more dropout layer and a dense layer which has a single node and the sigmoid activation function is included to complete the model construction. The sigmoid function

converts the input into a number between 0.0 and 1.0. The function has an S-shape from zero up through 0.5 to 1.0 for all potential inputs. The Adam optimization algorithm with a default learning rate of 0.001 was selected. A binary cross entropy loss function was selected. The batch size was 10.

3.3 Image-classification model: CNN Model

To detect spam with images we have selected the CNN model. A convolutional neural network (CNN) is used for processing data that has a grid-like architecture, such as image [36]. They are particularly effective for image classification and object detection tasks. CNNs consist of a series of layers. A series of filters are applied to the input data by the convolutional layers in order to extract features like edges and patterns. In order to simplify the network's computations and increase its adaptability to fresh data, the pooling layers shrink the spatial size of the feature maps. The fully-connected layers connect all of the features extracted by the convolutional layers and use them to make a prediction [37].

3.3.1 Mathematics of CNN Model

The core mathematics behind CNNs involves the use of matrix operations to compute the output of each layer in the network. The technique involves altering the image depending on values from a tiny number matrix known as the kernel or filter. The following formula (Fig. 2) yields the values of the subsequent feature map, where the input picture is denoted by letter f and our kernel by letter h. The integers m and n, respectively, are used to represent the rows and columns of the result matrix's indexes. Fig. 2 represents the Feature Map Formula.

$$G[m,n] = (f * h)[m,n] = \sum_{j} \sum_{k} h[j,k] f[m-j, n-k]$$

Fig. 2: Feature Map Formula

Convolution over volume enables the application of multiple filters within a single layer. Convolution (Fig. 3) is carried out individually and later the results are stacked. The dimensions of the received tensor meet the following equation, in which: n denotes the image size, f denotes the filter size, nc denotes the number of channels in the image, p denotes the used padding, s denotes the used stride, nf denotes the number of filters. Fig. 3 represents the Convolution Formula.

$$[n, n, n_c] * [f, f, n_c] = \left[\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor, \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor, n_f \right]$$

Fig. 3: Convolution formula

In addition to convolution, CNNs also involve the use of activation functions, such as ReLU or sigmoid, which are applied element-wise to the output of each layer. Activation functions are used to introduce non-linearity into the model, which is important for learning more complex relationships in the data. CNNs also involve the use of pooling layers, which are used to down sample the data and reduce the dimensionality of the feature maps [38].

3.3.2 Image dataset

Similar to the LSTM model a 5-fold cross-validation technique was utilized to obtain the results from the CNN model. The input dataset consisted a total of 37,060 entries. The entire dataset was sourced from various websites figure 6 is a sample of the images used as ham and figure 7 is a sample of the images used as spam. The entire dataset was divided into 5 folds and a train-test validation split of 0.2 was maintained each time. When fold 1 was used a test dataset it consisted of 7,414 entries and the remaining 29,646 entries were grouped into the remaining folds and were used as training datasets. This process was continued when folds 2, 3 and 4 were used as test datasets respectively. When fold 5 was used a test dataset it consisted of 7,404 entries and the remaining 29,656 entries were grouped into the remaining folds and were used as training datasets. Fig. 4 depicts some image ham dataset samples and Fig. 5 depicts some image spam samples.

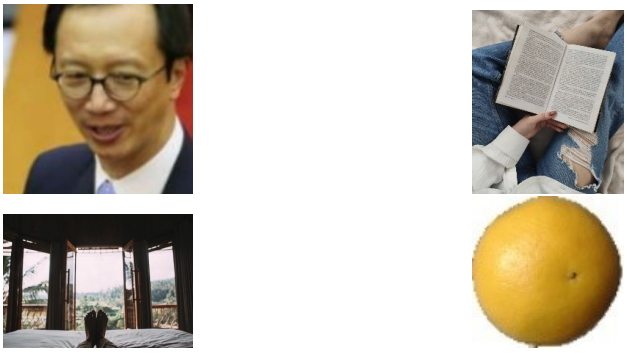


Fig. 4: Image ham dataset samples

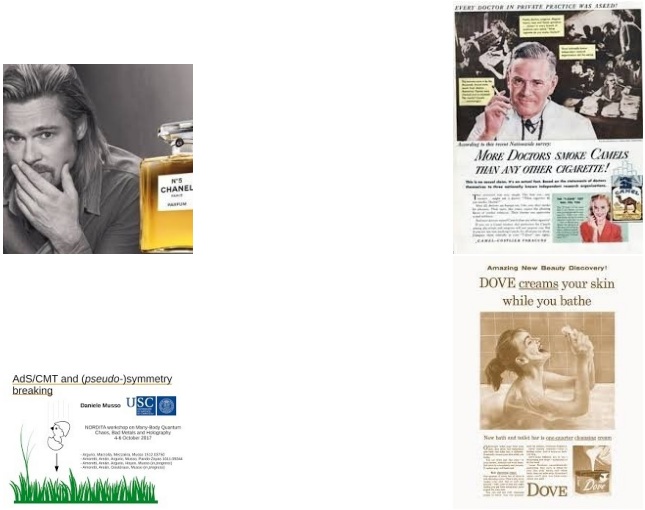


Fig. 5: Image spam dataset samples

3.3.3 Image pre-processing

After collecting data with images it has to be pre-processed. This is done to prepare the image data into a format that can be efficiently processed by the CNN model. The following steps were used for pre-processing the image data. Samples with corrupted data was removed. All images were converted to the JPG format. The images were resized to a consistent size (224 pixels by 224 pixels). By applying data augmentation techniques like rotation, translation, and flipping to the photos, it was possible to expand the training dataset and strengthen the model's robustness. The classed and labelled final photos.

3.3.4 Architecture of CNN Model:

A pre-built VGG (visual geometry group) model was utilized.VGG19 is a convolutional neural network (CNN) model that was trained on the ImageNet dataset for image classification tasks. There are 19 layers total in the VGG19 model, comprising 6 totally linked layers and 13 convolutional layers. The fully-connected layers are utilised for classification, while the convolutional layers are in charge of extracting information from the input image. It has received top scores on a number of benchmarks and has been widely utilised as a basic model for various image classification and object recognition applications.

The first layer has an input shape of (128,128) the input image is of RGB nature, the layer has a total of 32 filters, a kernel size of 3 which specifies the size of these convolutional filters is employed. Next is a batch normalization layer and a 'Relu' activation layer, a maxpooling layer with a pool size of 2 and number of strides 2 is now added onto the model

By pooling layers, the feature maps' size is decreased. Therefore, it requires fewer parameters to be learnt and requires less network processing. The pooling layer summarises the characteristics in an area of the feature map produced by the convolution layer. Therefore, instead of precisely positioned features produced by the convolution layer, further operations are conducted on summarised features. Due to this, the model is has become more resistant to changes in the features' positions in the input image. A two-dimensional filter is slid over each channel of the feature map during the pooling operation, and the features contained within the region covered by the filter are summarised. For a feature map having dimensions mh x mw x mc, the dimensions of output obtained after a pooling layer is

(mh - c + 1) / s x (mw - c + 1)/s x mc

where,

mh -> height of feature map

mw -> width of feature map

mc -> number of channels in the feature map

c -> size of filter

s -> stride length

For the current layer the values of the above parameters are :

mh -> 128

mw -> 128

mc -> 3

c -> 2

s -> 2

The maximum element from the area of the feature map covered by the filter is chosen using max pooling. As a result, the result would be a feature map with the standout elements from the prior feature map. The output shape from the maxpool layer is (64,64) and of rgb nature, this is now the input shape of the second 2d convolutional layer with the same hyper parameter values for number of filters and kernel size as the first convolutional layer. Next is a batch normalisation layer, the 'Relu' activation layer and maxpooling layer with the same hyperparameters as the previous layer. A third 2d convolutional layer is next and after it is the full suite of layers as mentioned above similar to the previous convolutional layers. The input shape to this 3rd layer is (32,32) and is of RGB type .

As observed the size of input to each convolutional layer halves due to the presence of the maxpool layer connected before each convolutional layer, this architecture allows the model to extract the most prominent features and even robustly judge the type of features at varying angles and varying sizes. The output from the final 2d maxpool layer is finally flattened and given into a dense layer with 64 nodes, the output from

this layer is then batch normalized and the 'Relu' activation function is applied to it, the output is again fed into another dense layer this time with 32 nodes followed by batch normalization and application of the activation function.
The final layer to the CNN image classification model is a dense layer with 1 node and the sigmoid activation function which guarantees that the output of this layer will always be between 0 and 1, this is a probability value which predicts the presence of junk/spam information in the initial input image. Table 2 depicts the architecture of the CNN Model used.

Table 2: CNN Model architecture

Layer (type)	**Output Shape**	**Number of Parameters**
input_1 (Input Layer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 512)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
global_max_pooling2d	(None, 512)	0
dense (Dense)	(None, 512)	262656
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32896
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 128)	16512
dropout_3 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 2)	258
Total params: 15,158,338		
Trainable params: 443,650		
Non-trainable params: 14,714,688		

A Sparse Categorical Cross entropy loss function along with a sigmoid activation layer was selected. The Adam optimization algorithm with a default learning rate of 0.001 was used. Each image input has a shape of (224, 224, 3) and is treated as a 3-dimensional entity. The batch size was 32.

4 Fusion Model

Our model classifies input data with text spam, image spam or a combination of both. It is essential that the model is able to handle all types of input. Our fusion model combines the predictions or decisions of multiple models to make a final prediction or decision. The goal of the fusion model is to improve the accuracy or robustness of the final prediction or decision by leveraging the strengths of multiple models. The stacking approach was used to create a meta-model which was trained to make the final prediction based on the predictions of the LSTM and CNN models.

4.1 Fusion model dataset:

For a given fold first the image spam probability is calculated then the text spam probability is calculated. After every 10 instances of image spam probabilities a 0.5 is introduced to indicate the absence of image in an email. Similarly it is done on the text side as well to indicate the absence of text in that email.
The two are then shuffled independently as pre-processing step. This is done to ensure that the model is not biased to the order of the data samples. They are then concatenated column wise (feature level fusion). First column is the image spam probability and the 2nd column is the probability of that text piece being spam and each entry represents an email (1 input). Table 3 depicts the input from feature level fusion sent to the fusion model.

Table 3: Fusion Model input from feature level fusion

Image Spam Probability	**Text Spam Probability**
0.89041	0.00012
0.26861	0.00255
0.46964	0.00160
0.74551	0.5
0.99889	0.00021
0.99943	0.00039
0.35573	0.00021
0.96874	0.5
0.99992	0.00034
0.45806	8.84045e-05
0.99017	0.01955
0.5	9.41850e-05
0.5	0.99965
0.24603	0.00101
0.42308	0.00241
0.86186	0.00524
0.5	0.00570
0.86689	0.00363

While iterating through the table if in any entry there exists 0.5 probability in both columns (indicating the absence of both image and text part) then it is removed. After

this process was completed a total of 1100 entries remained. A test list was created, and table 4 depicts such an example.

Table 4: Fusion Model test list

Image Processing	Text Processing
1	0
0	1
0	0
1	1
1	0
1	0
0	1
1	0
1	1
0	0

In this list, 1 indicates the presence of spam and 0 indicates the absence of spam. For the training of fusion model, in each fold 4456 entries are used. In each fold of fusion model test list there are 634 spam instances and 466 ham instances.

4.2 Architecture of Fusion Model:

Table 5 depicts the architecture of the fusion model. The Adam optimization algorithm with a default learning rate of 0.001 was selected. A binary cross entropy loss function was selected.

Table 5: Fusion Model Architecture

Layer (type)	Output Shape	Number of Parameters
Dense_31 (Dense)	(None, 512)	1536
Dense_32 (Dense)	(None, 256)	131328
Dense_33 (Dense)	(None, 128)	32896
Activation_10 (Activation)	(None, 128)	0
Dropout_22 (Dropout)	(None, 128)	0
Dense_34 (Dense)	(None, 64)	8256
Activation_11 (Activation)	(None, 64)	0
Dropout_23 (Dropout)	(None, 64)	0
Dense_35 (Dense)	(None, 1)	65
Total params: 174,081		
Trainable params: 174.081		
Non-trainable params: 0		

5 Results

5.1 LSTM Model Results

The LSTM model was made to run for 20 epochs. The model loss history and the model training history for each of the 5 folds can be viewed in the graphs below. This is followed by two tables which numerically show the model's performance based on the metrics such as accuracy, precision, recall for each of the 5 folds. The second table shows the average of these five folds and demonstrated the model's performance. Our model had an overall accuracy of 0.9858 and a precision of 0.9856. The following sub-sections provide a graphical representation of the LSTM Model loss and training history.

5.1.1 LSTM Model Loss

Fig. 6 depicts the model loss history for the LSTM model. As the number of epochs increase the loss for the train and validation sets both decrease over the course of training, both lowering to below 0.1. This indicates that the model is learning and becoming more accurate. The model was made to stop after 20 epochs. Similar plots were obtained when other folds were utilized as validation datasets.

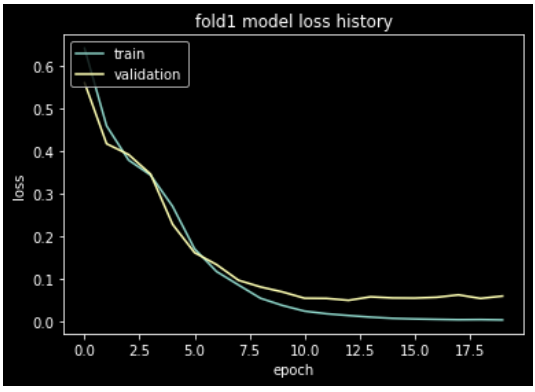


Fig. 6: LSTM Model Loss History (Fold 1)

5.1.2 LSTM Model Training History

Fig. 7 shows that the accuracy of the model on both the training set and the validation set increases over the course of training. This indicates that the model is learning and becoming more accurate. After 10 epochs the increase in accuracy is not exponential. The model trained for 20 epochs . Similar plots were obtained when other folds were used as validation datasets.

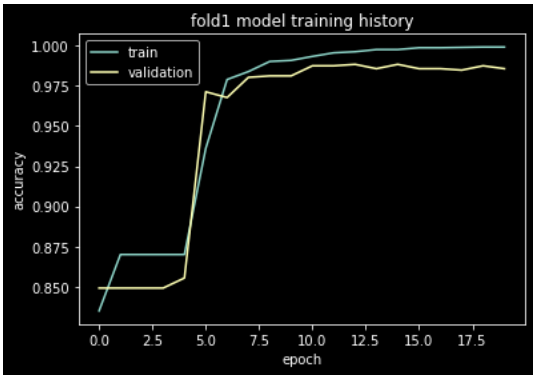


Fig. 7: LSTM Model Training History (Fold 1)

5.1.3 LSTM Model Performance

Tables 6 displays the performance of the model with respect to the different evaluation metrics and table 7 displays the performance of the model on average.

Table 6: LSTM Model performance for 5 folds

	ACCURACY	PRECISION	RECALL	F1 SCORE	ROC AUC	COHENs KAPPA
FOLD1	0.982944	0.986359	0.993658	0.989995	0.984100	0.933224
FOLD2	0.985637	0.985787	0.997945	0.991828	0.988221	0.932587
FOLD3	0.980251	0.984772	0.992835	0.988787	0.981912	0.906092
FOLD4	0.981149	0.981481	0.996865	0.989114	0.989114	0.918911
FOLD5	0.988299	0.989733	0.996898	0.993302	0.992962	0.947044

Table 7: LSTM model performance on average

ACCURACY	PRECISION	RECALL	F1 SCORE	ROC AUC	COHENs KAPPA
0.983656	0.985626	0.995640	0.990005	0.9872618	0.927517

5.2 CNN Model Results

The CNN model was made to run for 20 epochs. The model loss history and the model training history for each of the 5 folds can be viewed in the graphs below. This is followed by two tables which numerically show the performance of the model based on the metrics such as accuracy, precision, recall for each of the 5 folds. The second table shows the average of these five folds and demonstrated the model's performance as a whole. Our model had an overall accuracy of 0.9672 and a precision of 0.9689. The following sub-sections provide a graphical representation of the CNN Model loss and training history.

5.2.1 CNN Model Loss

Fig. 8 depicts the model loss history for the CNN model. The model is trained for a total 20 epochs , model loss drops off drastically in the initial training epochs. As the model approaches convergence the rate of decrease of loss is less. Similar plots were obtained when other folds were utilized as validation datasets.

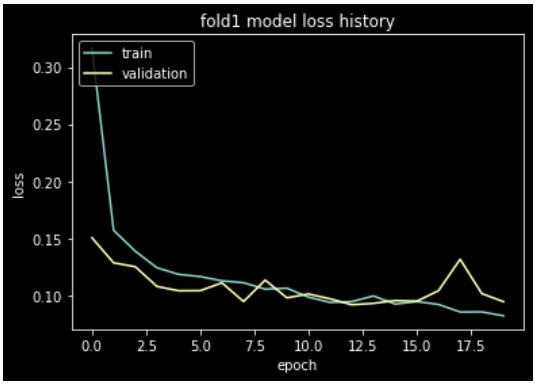


Fig. 8: CNN Model Loss History (Fold 1)

5.2.2 CNN Model Training History

Fig. 9 depicts the model training history for the CNN model. The model is trained for 20 epochs, the accuracy raises steeply and as the point of convergence is approached, increase in accuracy is reduced. Similar plots were obtained when other folds were utilized as validation datasets.



Fig. 9: CNN Model Training History (Fold 1)

5.2.3 CNN Model Performance

Tables 8 displays the performance of the model with respect to the different evaluation metrics and table 9 displays the performance of the model on average.

Table 8: CNN Model performance for 5 folds

	ACCURACY	PRECISION	RECALL	F1 SCORE	ROC AUC	COHENs KAPPA
FOLD1	0.96898	0.96735	0.97864	0.97302	0.00475	0.93653
FOLD2	0.96763	0.97168	0.97168	0.97168	0.00449	0.93390
FOLD3	0.96655	0.96202	0.98019	0.97101	0.00410	0.93148
FOLD4	0.96575	0.97931	0.96036	0.96974	0.00455	0.93327
FOLD5	0.96731	0.96415	0.97919	0.97162	0.00473	0.93310

Table 9: CNN model performance on average

ACCURACY	PRECISION	RECALL	F1 SCORE	ROC AUC	COHENs KAPPA
0.967242	0.968903	0.974037	0.971416	0.004508	0.933057

5.3 Fusion Model Results

The fusion model was made to run for 32 epochs. The model accuracy, model loss and the ROC curves can be viewed in the subsections below. This is followed by two tables which numerically show the performance of the model based on the metrics such as accuracy, precision, recall for each of the 5 folds. The second table shows the average of these five folds and demonstrated the model's performance as a whole. Our model had an overall accuracy of 0.954 and a precision of 0.9412.

5.3.1 Model Loss

Fig. 10 depicts the model loss history for the fusion model. Similar plots were obtained when other folds were utilized as validation datasets.

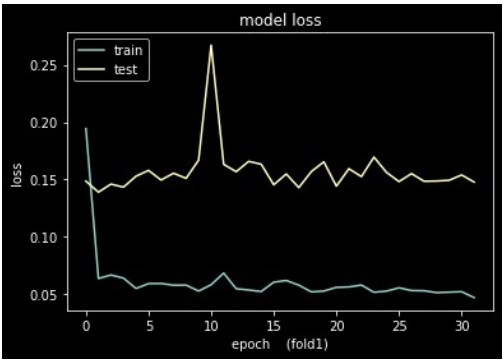


Fig. 10: Model Loss (Fold 1)

5.3.2 Model Accuracy

Fig. 11 depicts the model training history. Similar plots were obtained when other folds were utilized as validation datasets.

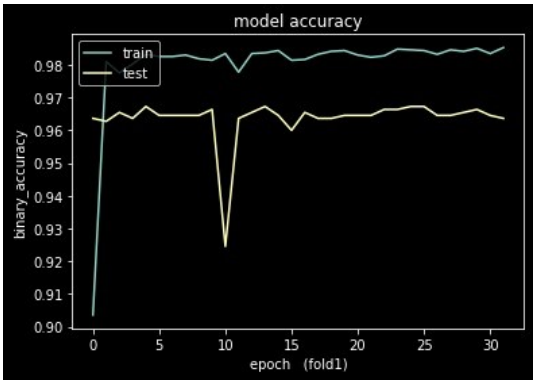


Fig. 11: Model Training History (Fold 1)

5.3.3 Fusion Model Performance

Tables 10 displays the performance of the model with respect to the different evaluation metrics and table 11 displays the performance of the model on average.

Table 10: Fusion Model performance for 5 folds

	ACCURACY	PRECISION	RECALL	F1 SCORE
FOLD1	0.964384	0.963048	0.975573	0.987340
FOLD2	0.935513	0.950893	0.964856	0.944488
FOLD3	0.960802	0.929978	0.951589	0.966949
FOLD4	0.955576	0.926441	0.940800	0.960000
FOLD5	0.955535	0.935982	0.955927	0.962510

Table 11: Fusion model performance on average

ACCURACY	PRECISION	RECALL	F1 SCORE
0.954362	0.941268	0.957749	0.960866

5.4 ROC Curve

Fig. 12 depicts the ROC curve of the model when Fold 1 is selected as the testing dataset. The ROC curve in the image shows the performance of the model in detecting spam. The curve plots the true positive rate (TPR) against the false positive rate (FPR) for different thresholds of the model's predictions. The TPR is the fraction of spam messages that are correctly identified as spam. The FPR is the fraction of non-spam messages that are incorrectly identified as spam. The model is good at identifying spam messages, while also minimizing the number of non-spam messages that are incorrectly identified as spam.

The ROC curve also shows that the model performs better than a "no skill" classifier, which would simply classify all emails as either spam or non-spam randomly. This means that the deep learning model is able to learn to identify spam emails based on features of the emails that are not obvious to humans. The AUC (area under the ROC curve) is a measure of how well a classifier performs overall. An AUC of 1.0 indicates a perfect classifier, while an AUC of 0.5 indicates a classifier that is no better than random guessing. The AUC for this deep learning project is 0.95, which is very high. This indicates that the model is able to distinguish between spam and non-spam emails with a high degree of accuracy.

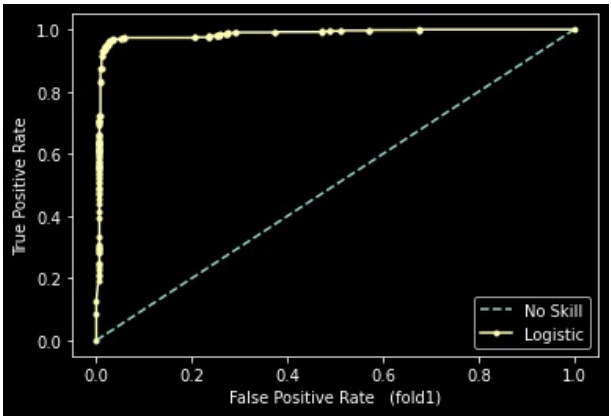


Fig. 12: ROC Curve for Fold 1

5.5 Comparative analysis

Table 12 shows the performance of the proposed model in comparison to other methods in the are of spam detection. The methodology, multi modality and accuracy were primarily taken into account for comparison

Table 12: Performance comparison of existing spam detection methods

Tool	Methodology	Multimodality	Evaluation metrics
ESM-S [39]	Employs the average of the probabilities of the class predictions from various classifier using a soft voting method.	No, does not analyse potential image spam in emails and hence spam can bypass model	Average Accuracy Text spam = 94.06%
Random Forest [40]	Random forest machine learning algorithm.	No, does not analyse potential image spam in emails and hence spam can bypass model	Average Accuracy Text spam = 94.50%
ANN [41]	Classification of image spam by considering statistical image feature histogram and mean value of an block of image	No, does not analyse potential text spam in emails and hence spam can bypass model	Average Accuracy Image spam = 92.82%
KNN [42]	Multiple features fusion method	No, does not analyse potential text spam in emails and hence spam can bypass model	Average Accuracy Image spam = 94.00%
GA and RWN [43]	Spam identified by analysing relevant features based on evolutionary networks	Yes, analyses both text and image content. Email content containing a combination of both cannot bypass model.	Average accuracy Combination of both = 92.00%
WC Based algorithm [44]	Hybrid Water Cycle Optimization Algorithm With Simulated Annealing	Yes, analyses both text and image content. Email content containing a combination of both cannot bypass model.	Average accuracy Combination of both = 94.60%
MDL-SPAM	Text and image spam in emails detected using LSTM model and CNN model respectively. Fusion model combines the outputs of both the models and gives final probability of spam.	Yes, analyses both text and image content. Email content containing a combination of both cannot bypass model.	Average Accuracy Combination of both = 95.43%

6 Conclusion

In this paper, an innovative approach to combat email spam is presented, addressing the contemporary challenge of spammers embedding content within images. The research employed a multifaceted strategy, utilizing a transfer learning model for image analysis and an LSTM model for text analysis, covering a broader spectrum of potential spam tactics. A notable feature of this approach is the use of a less biased dataset for training the image spam detection model, which includes images with a mixture of graphics and text, enhancing its adaptability to diverse spam content. The introduction of a fusion model, combining the results from both image and text-based spam detection models, significantly boosts the overall spam detection system's robustness. Impressively, this approach achieves an accuracy of 95.43% and a precision of 94.12%, showcasing its effectiveness in reducing the impact of spam emails and serving as a promising model for countering evolving spam tactics in the digital landscape.

Data Availability All datasets are publicly available here.

Declarations

Competing interests: The authors declare that they have no conflict of interest.
Funding: This research received no external funding.

References

[1] Imber, D.: The Latest Phishing Statistics (updated June 2023) AAG IT Support˝ (2023). https://aag-it.com/the-latest-phishing-statistics/

[2] 15 Outrageous Email Spam Statistics that Still Ring True in 2018. https://www.propellercrm.com/blog/email-spam-statistics

[3] Emailtray Blog. https://www.emailtray.com/blog/category/email-trends-and-stats/

[4] Cveticanin, N.: What039;s On the Other Side of Your Inbox - 20 SPAM Statistics for 2023 (2023). https://dataprot.net/statistics/spam-statistics/

[5] Moorthy, J.: 23 Email Spam Statistics to Know in 2023. Mailmodo (2023)

[6] The most common spam emails in 2022 (2023). https://selzy.com/en/blog/spam-email-examples/

[7] Kulikova, T.: Spam and phishing in 2020 (2021)

[8] Karim, A.M., Azam, S., Shanmugam, B., Kannoorpatti, K.: An Unsupervised Approach for Content-Based Clustering of Emails Into Spam and Ham Through

Multiangular Feature Formulation. IEEE Access **9**, 135186–135209 (2021). https://doi.org/10.1109/access.2021.3116128

[9] contributors, W.: Botnet. Wikipedia (2023)

[10] Gillis, A.S.: phishing. Security (2023)

[11] Jáñez-Martino, F., Alaiz-Rodríguez, R., González-Castro, V., Fidalgo, E., Alegre, E.: A review of spam email detection: analysis of spammer strategies and the dataset shift problem. Artificial Intelligence Review **56**(2), 1145–1173 (2022). https://doi.org/10.1007/s10462-022-10195-4

[12] Zola, A.: spam filter. Security (2021)

[13] Adewole, K.S., Anuar, N.B., Kamsin, A., Sangaiah, A.K.: Smsad: a framework for spam message and spam account detection. Multimedia Tools and Applications **78**, 3925–3960 (2019)

[14] Khurana, D., Koli, A., Khatter, K., Singh, S.: Natural language processing: State of the art, current trends and challenges. Multimedia tools and applications **82**(3), 3713–3744 (2023)

[15] Annadatha, A., Stamp, M.: Image spam analysis and detection. Journal of Computer Virology and Hacking Techniques **14**(1), 39–52 (2016). https://doi.org/10.1007/s11416-016-0287-x

[16] Kumaresan, T., Sanjushree, S., Suhasini, K., Palanisamy, C.: Image spam filtering using support vector machine and particle swarm optimization. Int. J. Comput. Appl **1**, 17–21 (2015)

[17] Abd Razak, S.B., Mohamad, A.F.B.: Identification of spam email based on information from email header. In: 2013 13th International Conference on Intellient Systems Design and Applications, pp. 347–353 (2013). IEEE

[18] Madisetty, S., Desarkar, M.S.: A neural network-based ensemble approach for spam detection in twitter. IEEE Transactions on Computational Social Systems **5**(4), 973–984 (2018)

[19] Salunkhe, A.: Attention-based Bidirectional LSTM for Deceptive Opinion Spam Classification (2021). http://arxiv.org/abs/2112.14789

[20] Hosseinalipour, A., Ghanbarzadeh, R.: A novel approach for spam detection using horse herd optimization algorithm. Neural Computing and Applications **34**(15), 13091–13105 (2022)

[21] Srinivasan, P., et al.: Tp-detect: trigram-pixel based vulnerability detection for ethereum smart contracts. Multimedia Tools and Applications, 1–15 (2023)

[22] JJ, L., Singh, K., Chakravarthi, B.: Digital forensic framework for smart contract vulnerabilities using ensemble models. Multimedia Tools and Applications, 1–44 (2023)

[23] Kumar, S., Asthana, R., Upadhyay, S., Upreti, N., Akbar, M.: Fake news detection using deep learning models: A novel approach. Transactions on Emerging Telecommunications Technologies **31**(2), 3767 (2020)

[24] Anayat, S., Ali, A., Ahmad, H.: Using a probable weight based bayesian approach for spam filtering. In: 8th International Multitopic Conference, 2004. Proceedings of INMIC 2004., pp. 340–345 (2004). IEEE

[25] Karim, A., Azam, S., Shanmugam, B., Kannoorpatti, K.: Efficient clustering of emails into spam and ham: The foundational study of a comprehensive unsupervised framework. IEEE Access **8**, 154759–154788 (2020)

[26] Alonso, O.: Challenges with label quality for supervised learning. Journal of Data and Information Quality (JDIQ) **6**(1), 1–3 (2015)

[27] Manlangit, S., Azam, S., Shanmugam, B., *et al.*: Novel machine learning approach for analyzing anonymous credit card fraud patterns. International Journal of Electronic Commerce Studies **10**(2), 175–202 (2019)

[28] Basavaraju, M., Prabhakar, D.R.: A novel method of spam mail detection using text based clustering approach. International Journal of Computer Applications **5**(4), 15–25 (2010)

[29] Ravindran, R.M., Thanamani, A.S.: K-means document clustering using vector space model. Bonfring International Journal of Data Mining **5**(2), 10–14 (2015)

[30] Laorden, C., Ugarte-Pedrero, X., Santos, I., Sanz, B., Nieves, J., Bringas, P.G.: Study on the effectiveness of anomaly detection for spam filtering. Information Sciences **277**, 421–444 (2014)

[31] Alsmadi, I., Alhami, I.: Clustering and classification of email contents. Journal of King Saud University-Computer and Information Sciences **27**(1), 46–57 (2015)

[32] Liu, R., Yang, N., Ding, X., Ma, L.: An unsupervised feature selection algorithm: Laplacian score combined with distance-based entropy measure. In: 2009 Third International Symposium on Intelligent Information Technology Application, vol. 3, pp. 65–68 (2009). IEEE

[33] Ma, R., Wang, Y., Cheng, L.: Feature selection on data stream via multi-cluster structure preservation. In: Proceedings of the 29th ACM International Conference on Information & Knowledge Management, pp. 1065–1074 (2020)

[34] Zhu, X., Wang, Y., Li, Y., Tan, Y., Wang, G., Song, Q.: A new unsupervised feature selection algorithm using similarity-based feature clustering. Computational Intelligence **35**(1), 2–22 (2019)

[35] Syakur, M.A., Khotimah, B.K., Rochman, E.M.S., Satoto, B.D.: Integration K-Means Clustering Method and Elbow Method For Identification of The Best Customer Profile Cluster. IOP conference series **336**, 012017 (2018). https://doi.org/10.1088/1757-899x/336/1/012017

[36] Faris, H., Alqatawna, J., Ala'M, A.-Z., Aljarah, I., *et al.*: Improving email spam detection using content based feature engineering approach. In: 2017 IEEE Jordan Conference on Applied Electrical Engineering and Computing Technologies (AEECT), pp. 1–6 (2017). IEEE

[37] DeBarr, D., Wechsler, H.: Using social network analysis for spam detection. In: Advances in Social Computing: Third International Conference on Social Computing, Behavioral Modeling, and Prediction, SBP 2010, Bethesda, MD, USA, March 30-31, 2010. Proceedings 3, pp. 62–69 (2010). Springer

[38] Faris, H., Aljarah, I., Al-Shboul, B.: A hybrid approach based on particle swarm optimization and random forests for e-mail spam filtering. In: Computational Collective Intelligence: 8th International Conference, ICCCI 2016, Halkidiki, Greece, September 28-30, 2016. Proceedings, Part I 8, pp. 498–508 (2016). Springer

[39] Oh, H.: A youtube spam comments detection scheme using cascaded ensemble machine learning model. IEEE Access **9**, 144121–144128 (2021)

[40] Choudhary, N., Jain, A.K.: Towards filtering of sms spam messages using machine learning based technique. In: Advanced Informatics for Computing Research: First International Conference, ICAICR 2017, Jalandhar, India, March 17–18, 2017, Revised Selected Papers, pp. 18–30 (2017). Springer

[41] Soranamageswari, M., Meena, C.: Statistical feature extraction for classification of image spam using artificial neural networks. In: 2010 Second International Conference on Machine Learning and Computing, pp. 101–105 (2010). IEEE

[42] Yuan, S., Zhang, C.: An improved multiple features fusion method for image spam filtering. In: 2016 3rd International Conference on Information Science and Control Engineering (ICISCE), pp. 200–203 (2016). IEEE

[43] Faris, H., Ala'M, A.-Z., Heidari, A.A., Aljarah, I., Mafarja, M., Hassonah, M.A., Fujita, H.: An intelligent system for spam detection and identification of the most relevant features based on evolutionary random weight networks. Information Fusion **48**, 67–83 (2019)

[44] Al-Rawashdeh, G., Mamat, R., Abd Rahim, N.H.B.: Hybrid water cycle optimization algorithm with simulated annealing for spam e-mail detection. IEEE Access **7**, 143721–143734 (2019)